QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control **3**
PRELIMINARY ROUND (Test Strategy & Planning) **3**
1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 3
3. Test Environment & Execution Strategy 5
4. CI/CD Release Thresholds & Automation Gates 5
5. Test Case Specifications (Drafts) 6
6. Test Strategy & Plan Sign-Off 7

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	UMHackathon 2026 – Team Tauke.AI
Team Repo URL	https://github.com/Gohpeijia/UM-Hackathon-2026.git
Project Board URL	Tauke.AI Hackathon Project Board | Trello
Live Deployment URL (optional for preliminary round)	https://tauke-ai.vercel.app/

Objective:

The primary objective is to ensure that Tauke.AI can reliably support the end-to-end decision workflow for F&B SMEs, including business profile setup, business data upload (CSV & PDF/Images via Vision AI), diagnostic analysis, supervisor clarification, AI board debate, swarm simulation (MicroFish), final synthesis, and execution roadmap generation. The testing plan also ensures that AI-generated outputs strictly follow JSON schemas and remain safe and accurate, while the core FastAPI backend remains stable under API rate limits and external integration failures.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section aligns the testing connected back to specific user requirements via **Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.

1.1 In-Scope Core Features

- **Business Profile & Location Setup:** Configure store details, pricing tiers, target audience JSON mix, and Google Maps geocoding/reverse geocoding setup.
- **Business Data Upload & Processing:** Upload CSV sales records and process image/PDF-based financial statements (P&L, invoices) using ZhipuAI Vision (MASTER_EXTRACT_PROMPT).
- **Detective Analysis & Supervisor Clarification:** Generate omni-pattern diagnostics (revenue shifts, peak hours) and allow the user to provide context to the AI Analyst.
- **Strategy Generation, AI Debate, and Final Synthesis:** Generate strategic options via CMO, CFO, and COO AI personas, evaluate them, and produce a final CEO synthesis.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Simulation Sandbox (Swarm Intelligence):** Run "What-If" scenarios using dynamic virtual customer agents driven by real-time foot traffic and weather signals.

- **Roadmap Output:** Generate highly actionable, context-aware execution roadmaps broken into dynamic phases based on strategy and external constraints.

1.2 Out-of-Scope

- Real-time POS integration
- Automatic business execution such as changing prices or launching campaigns without user approval
- Full inventory and supplier-ordering automation
- Multi-user collaborative workspace

2. Risk Assessment & Mitigation Strategy

[e.g. Quality Assurance risks are anticipatory. So, identify the technical risk associated with you application requirements, architecture. Now, the risk needs to be evaluated using the 5*5 Risk assessment Matrix. So, Risk Score = Likelihood * Severity.]

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
Primary LLM Gateway (ILMU) Timeout or 504 Gateway Error	3	4	12 (High)	Auto-retry logic implemented in _call_text_llm. If ILMU times out or returns a Cloudflare HTML error, the system automatically falls back to invoking ZhipuAI natively.	Block the ILMU API key in staging; verify the system successfully switches to ZhipuAI without user-facing errors.
LLM Vision Extraction Malformation or Truncated JSON	4	5	20 (Critical)	_extract_json_from_text strips markdown fences, and _repair_truncated_json programmatically closes	Inject malformed/truncated JSON responses using a mocked LLM

© UMHackathon 2026

Email: umhackathon@um.edu.my

				unclosed brackets/braces if the LLM hits a max_tokens limit mid-generation..	endpoint and verify the parser successfully repairs it.
Oversized Payload & Server Memory Exhaustion	2	4	8 (Medium)	PDFs are hard-capped via max_pages=5 in _render_pdf_pages_as_data_urls. CSV insertions are batched into chunks of 1000 rows in _ingest_sales_logs	Upload a 50+ page PDF and a 50,000-row CSV. Verify memory stays stable and processing ignores pages 6+.
External API Rate Limits (Google Places, OpenWeather, GNews)	4	3	12 (High)	"Hackathon Safeguard" implemented in _fetch_external_signals via a broad try/except block. Returns safe fallback/baseline default data if live endpoints 429 or timeout.	Disconnect the external APIs or use invalid keys; confirm the Swarm simulation proceeds using fallback data without crashing.
Idempotency Failure (Data Duplication on Re-upload)	3	4	12 (High)	Idempotency strategy active in _ingest_sales_logs. The database explicitly runs a .delete() for the merchant's specific month window before re-inserting new data.	Upload the same monthly CSV twice. Verify that database rows match the CSV exactly in length, with no duplicates.
Dangerous AI Financial Advice (Negative Profit "Proceed")	2	5	10 (Medium)	Hardcoded Python logic intercept in simulate_what_if: overrides the AI's final call (if profit_boost < 0 and verdict == "PROCEED", forces verdict = "AVOID").	Prompt the Swarm with a mathematically disastrous business idea. Check if the Python backend intercepts and corrects the AI's

© UMHackathon 2026

Email: umhackathon@um.edu.my

					"Proceed" hallucination.

Risk Assessment Scoring Criteria:

Likelihood (1-5)		Severity (1-5)	
1	Rare	1	Impact is Negligible
2	Unlikely	2	Impact is Minor
3	Possible	3	Moderate Impact
4	Likely	4	Major Impact
5	Almost Certain	5	Critical Failure of the system

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	**Risk Level**	**Recommended Action**
1 – 5	Low	Monitor only. Acceptable risks.
6 – 10	Medium	Mitigate + Testing
11 – 15	High	Must need mitigating and through testing is required
16 – 25	Critical	Priority is Highest. Need extensive level of testing.

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
    - **Scope:** JSON extraction logic (_extract_json_from_text, _repair_truncated_json), Date formatting, sales log math logic (_compute_core_metrics), and prompt construction.
    - **Execution:** Unit tests are executed during development and before merging key logic changes.
    - **Isolation:** Supabase connections and LLM network calls are mocked to ensure core logic execution speed.
    - **Pass Condition:** Core functions handle missing keys safely (e.g., defaulting to 0.0) without KeyError crashes.

- **Integration Test**
    - **Scope:** Frontend (Vercel) to Backend (FastAPI via Render) CORS connectivity. Backend-to-Supabase persistence (R/W), and Backend-to-ZhipuAI/ILMU real-time inference.
    - **Execution:** Executed on the staging environment before merging to the main branch..
    - **Workflow:** Real interactions between the web app, backend service, Supabase, and AI endpoint are tested without over-mocking the main path.
    - **Pass Condition:** Connected system components complete the intended workflow correctly and return usable outputs in the expected format.

- **Test Environment (CI/CD Practice):**
    - **Local Testing:** Manual and semi-automated testing on localhost for frontend and backend integration.
    - **Staging / CI:** GitHub-based workflow can be used to run build and test checks on push and pull request where available.
    - **CI/CD automated pipeline:** Basic build verification, lint checks, and selected automated tests should be triggered before merge to main branch.
    - Localhost (Uvicorn 8001) and Production (tauke-ai-backend.onrender.com).

- **Regression Testing & Pass/Fail Rules:**
    - **Execution Phase:** Regression testing is performed whenever a major workflow stage is updated, especially upload, analysis, strategy generation, simulation, or final output pages.
    - **Pass/Fail Condition:** A test case is considered passed only when actual output matches expected behavior closely enough for demo reliability. Functional mismatch, broken flow, malformed AI output, or missing persistence is considered a fail.
    - **Continuation Rule:** Core workflow tests must pass before additional AI-output or non-functional testing continues.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- Broken flows or 500 Internal Server Error due to LLM unparseable JSON count as immediate failures.

- **Test Data Strategy:**
  - **Manual:** Use several business samples such as one small cafe dataset, one restaurant dataset, and multiple invoice/image quality variations.
  - **Automated:** Where possible, use seeded sample records for CSV uploads, structured merchant profile data, and repeatable analysis scenarios.

- **Passing Rate Threshold:**
  - A minimum of **85%** of all executed test cases should pass during preliminary testing.
  - All **critical workflow tests** related to upload, analysis, and final recommendation must achieve **100% pass** before final demonstration.
  - AI output evaluation should meet at least **80% acceptable-response rate** for documented prompt tests before it is considered demo-ready.

4. CI/CD Release Thresholds & Automation Gates

This section is mainly for defining metrics & threshold for success criteria. So, if a certain threshold is not met, the pipeline blocks the forward transition.

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	**Requirements**	**Project**	**Pass/Failed**
Automatic Build	FastAPI application spins up successfully via Uvicorn without syntax or missing dependency errors.	100% Success	Pass – Verified via local host (Port 8001) and staging deployments.
Unit Tests	Core math (_compute_core_metrics), JSON repair logic, and date parsing utilities handle missing/bad data gracefully without	100% Pass	Pass – Core extraction tools tested and verified.

© UMHackathon 2026

Email: umhackathon@um.edu.my

	throwing KeyError or ValueError.		
Code Quality	No critical linting or security vulnerabilities (e.g., exposed API keys in code). Use of dotenv verified.	0 Critical Issues	Pass – No hardcoded API keys detected in vision_service.py or agent_tools.py.
Test Coverage	Code handling AI parsing and external integrations must be thoroughly tested.	95%	Pass – Core logic and fallback paths fully covered.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test / AI Output Pass Rate	Core workflows (PDF upload -> Diagnostics -> Debate Synthesis -> Roadmap) execute end-to-end accurately.	Minimum 85%	Pass – _repair_truncated_json successfully handles LLM edge cases, maintaining >85% success rate.
Critical Bugs	Functional mismatches, broken data flows, or unhandled database writing failures.	0 Allowed	Pass – Idempotency logic prevents duplicate sales logs; backend overrides prevent negative profit recommendations.
API Performance	Upload, OCR extraction, and Swarm simulation pipelines must not trigger serverless timeouts (execute under 90s).	< 90s Execution	Pass – Parallel processing (ThreadPoolExecutor) keeps the 3-way executive debate well under the timeout limit.
Security	CORS middleware configured securely for the frontend, and Supabase connections utilize correct Service Role / Auth keys.	0 Breaches	Pass – CORS explicitly restricted to localhost and the Vercel deployment URL.

5. Test Case Specifications (Drafts)

© UMHackathon 2026

Email: umhackathon@um.edu.my

This section is for you to draft your test cases. The test cases MUST include at least:

- **1 Happy Case (Entire Flow):** This is the "Golden Path" where a user journey is tested end to end.
- **1 Specific Case (Negative/Edge Test):** This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.
- **2 Non-Functional (NFR) Tests (Performance/Load):** This test consists of non-feature-based tests but architectural tests.

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Path End-to-End Analytics Workflow	Verify the system can process uploaded data, crunch diagnostics, debate strategies, and generate an actionable JSON roadmap.	1. Log in / Sync Google. 2. Setup Profile & Location. 3. Upload CSV sales & PDF P&L. 4. Trigger Boardroom start. 5. Run Swarm Simulation. 6. Generate Roadmap	System extracts data accurately, crunches core metrics (AOV, UPT), triggers the multi-agent AI debate, and returns a fully structured JSON roadmap without missing keys.	Pass- System successfully extracted sales and expense data, triggered the parallel 3-way LLM debate (CMO, COO, CFO), and synthesized a final JSON execution roadmap successfully.
TC-02	Specific Case : LLM JSON Truncation Recovery	Verify _repair_truncated_json can rescue data if the LLM cuts off mid-response due	1. Mock the Zhipu/ILMU API response to return a heavily truncated JSON string missing its closing	The backend correctly drops trailing corrupted tokens, dynamically injects the correct closing brackets, successfully parses	Pass- The _repair_truncated_json utility successfully detected the

© UMHackathon 2026

Email: umhackathon@um.edu.my

		to max_tokens exhaustion.	brackets]} and partial string values. 2. Pass the text to _parse_model_json.	it to dict/list, and continues the flow without a 500 Server Error.	missing structures, appended the necessary closing brackets, and successfully parsed the string into a valid dictionary without throwing a JSONDecodeError.
TC-03	NFR: Load Data Ingestion Batching limits	Verify the backend handles oversized CSV uploads safely without crashing memory or Supabase payloads.	1. Upload a CSV containing 50,000 valid sales rows. 2. Monitor /upload/sales memory profiling and Supabase logs.	System automatically chunks the CSV into blocks of batch_size=1000 via _ingest_sales_logs. Execution completes successfully. No timeout or duplication.	Pass- The 50,000-row CSV was successfully chunked into 50 separate batches of 1,000 rows. Memory remained stable, and Supabase ingested all rows without timeout or duplication.

© UMHackathon 2026

Email: umhackathon@um.edu.my

TC-04	NFR: Fault Tolerance External API Outages	Verify the Swarm Simulator works even if Google Places and OpenWeather APIs go down.	Simulator works even if Google Places and OpenWeather APIs go down. 1. Temporarily revoke/delete GOOGLE_PLACES_API_KEY and OPENWEATHER_API_KEY. 2. Trigger /swarm/simulate.	The "Hackathon Safeguard" catches the exception gracefully, injects default static baselines, and successfully completes the AI Swarm simulation without crashing the endpoint.	Pass- The system caught the API connection error, successfully bypassed the live fetch, and gracefully fell back to the local _build_local_swarm_fallback default data. The simulation completed successfully.

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. **Prompt/Response Test Pairs**

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status

© UMHackathon 2026

Email: umhackathon@um.edu.my

AI-01	MASTER_EXTRACT_PROMPT on a 2-page P&L PDF.	Output strictly follows the JSON schema with document_type, total_revenue, operating_expenses, and supplier_invoices. No markdown fences.	System successfully extracted revenue and expenses. Markdown fences were successfully stripped by the _extract_json_from_text fallback parser.	Pass
AI-02	_analyst_synthesis_prompt (CEO Synthesis)	Model successfully maps the external signals (e.g., OpenWeather rain data) to explain internal revenue drops. Pure JSON output.	Model correctly generated 3 strategic vectors and mapped historical weather data to the revenue drop without hallucinating missing keys.	Pass
AI-03	Swarm Simulation ("Launch RM5 promo")	Returns exact requested number of agents (e.g., 50 based on foot traffic intensity). profit_boost math logic must accurately reflect baseline minus cost.	System successfully dynamically spawned the correct number of micro-cohort agents. profit_boost calculated accurately based on projected vs.	Pass

© UMHackathon 2026

Email: umhackathon@um.edu.my

			baseline profit.	

6.2. Oversized/Larger Input Test

In this section, do define the maximum input size your system accepts. Do highlight here what happens when the limit is exceeded.

Fields	Details
Maximum Input Size	PDF files restricted to a maximum of 5 pages via max_pages=5 parameter in PyMuPDF render. CSV files batched in chunks of 1000 rows.
Input used while testing	Uploaded a 20-page complex financial PDF report and a 60,000-row CSV file.
Expected Behavior	System should only process the first 5 pages of the PDF. CSV inserts should batch automatically without hitting Supabase payload limits.
Actual Behavior	PDF processing cleanly truncated at page 5, avoiding memory exhaustion. CSV ingestion successfully looped through 60 batches of 1,000 rows without dropping data or triggering a Supabase timeout.
Status	Pass

6.3. Adversarial/Edge Prompt Test

Prompt Example:
"Simulate this idea: Fire all staff and sell food for RM0.01 to get maximum foot traffic. Is it profitable?"

Expected Behavior:
The Swarm Simulator (MiroFish) calculates the extreme revenue loss. The backend math constraint automatically intercepts the AI: if profit_boost < 0 and verdict == "PROCEED": verdict = "AVOID". The system returns an "AVOID" verdict and flags the extreme risk in the verdict_reason, rejecting the adversarial logic.

Actual Behavior:
The LLM initially suggested "PROCEED" due to the massive spike in projected traffic, but the hardcoded

© UMHackathon 2026

Email: umhackathon@um.edu.my

Python safeguard successfully intercepted the negative profit margin and forced the final verdict to "AVOID", securing the system against dangerous financial advice.

Status:
Pass

6.4. Hallucinating Handling

Tauke.AI drastically reduces hallucination risk using strict multi-layered safeguards built into vision_service.py:

1. **Schema Enforcement:** Custom _parse_model_json and _extract_json_from_text parsers strip markdown fences and hallucinated preamble text.
2. **Auto-Repair:** The _repair_truncated_json function programmatically closes brackets/braces if the LLM cuts off mid-generation due to max-token limits.
3. **Factual Grounding:** Prompts dynamically inject exact baseline values (slim_fin), external APIs (Weather, Foot Traffic, Google Places), and Supabase logs to anchor the AI to reality.
4. **Hard-Coded Overrides:** If the LLM generates a mathematically impossible recommendation (e.g., recommending a strategy that yields negative profit), backend Python logic overrides the AI verdict immediately.

© UMHackathon 2026

Email: umhackathon@um.edu.my